

MASTER INGENIERIE DES DONNÉES

Cours : Architecture Parallèle

DOCUMENTATION TECHNIQUE ET SCIENTIFIQUE

Analyse Comparative de Scalabilité Forte/Faible et
Optimisation Hiérarchique du Produit Matriciel via
OpenMP

*Évaluation Expérimentale, Métrologie des Caches et Pipeline de
Restructuration des Boucles*

Réalisé par :
Saint-Vil RENÉ

Remis le 27 Juin 2026

Enseignant : Jean Andris Adam (jean_andris.adam@ueh.edu.ht)

Table des matières

1	Introduction et Objectifs du Projet	4
2	Architecture Modulaire et Flux d'Exécution	4
2.1	Journalisation des Sorties Applicatives	4
3	Modélisation de la Localité et Hiérarchie Mémoire	9
3.1	Algorithme Naïf vs Inversion i - k - j	9
3.2	Équation du Cache-Blocking	9
4	Analyse Analytique de la Scalabilité Forte	9
4.1	Données Expérimentales Brutes	9
4.2	Visualisations et Interprétations Physiques	10
4.2.1	Analyse de la Super-linéarité ($E_p > 1$)	11
4.2.2	L'impact des limites de l'Hyper-Threading	11
5	Analyse Analytique de la Scalabilité Faible	11
5.1	Données Expérimentales Brutes	12
5.2	Visualisations et Interprétations Physiques	12
5.2.1	Analyse de la saturation de la bande passante mémoire	13
6	Conclusion	14

Table des figures

1	1- Journalisation textuelle de l'exécution et de la validation numérique croisée.	5
2	2- Journalisation textuelle de l'exécution et de la validation numérique croisée.	6
3	3- Journalisation textuelle de l'exécution et de la validation numérique croisée.	7
4	4- Journalisation textuelle de l'exécution et de la validation numérique croisée.	8
5	Temps d'exécution en scalabilité forte : On observe une décroissance hyperbolique stricte du temps de calcul parallèle, confirmant l'efficacité de la distribution OpenMP sur la charge fixe de $N = 1000$	10
6	Speedup en scalabilité forte : Représentation de l'accélération relative. La courbe expérimentale se positionne au-dessus de la droite idéale $y = p$ (Amdahl) entre 1 et 4 threads, matérialisant l'effet de super-linéarité. . . .	10
7	Efficacité en scalabilité forte : L'efficacité démarre à un niveau exceptionnel de 122% pour 2 cœurs avant de décroître sous la barre des 100% au-delà de 4 cœurs à cause des surcoûts matériels.	11
8	Temps d'exécution en scalabilité faible : Illustration de l'explosion des temps due à la croissance polynomiale de la charge de travail globale ($N = 8000$ à 8 threads).	12
9	Speedup en scalabilité faible : Évolution du rapport d'accélération calculé sur les échelles de tailles croissantes.	13
10	Efficacité en scalabilité faible : Traduit l'impact de la saturation de la bande passante du bus mémoire système lors du traitement des très grandes matrices.	13

1 Introduction et Objectifs du Projet

L'opération de produit matriciel carré $C = A \times B$ ($A, B, C \in \mathbb{R}^{N \times N}$) constitue l'un des piliers du calcul à haute performance (HPC). Bien que triviale dans sa formulation mathématique initiale, son exécution efficace se heurte sur les architectures modernes au goulot d'étranglement de la mémoire (*Memory Wall*).

Ce projet met en évidence les gains fondamentaux apportés par la parallélisation sur architecture à mémoire partagée via ****OpenMP**** conjuguée à une restructuration algorithmique par blocs (****Cache-Blocking**** ou ***Tiling***). L'objectif est d'étudier de manière quantitative le comportement de l'application selon deux paradigmes de scalabilité : forte et faible.

2 Architecture Modulaire et Flux d'Exécution

Le projet adopte une structure logicielle stricte pour isoler les composants :

- `include/matrix.h` : Fichier de configuration global contenant les macros mathématiques telles que `MIN` et les prototypes des noyaux.
- `src/matrix.c` : Prise en charge de bas niveau de la topologie mémoire (allocation dynamique unidimensionnelle contiguë pour maximiser la localité spatiale et empêcher la fragmentation).
- `src/sequential.c` : Version naïve triple boucle i - j - k servant de base de calibration temporelle.
- `src/openmp_naive.c` : Implémentation parallèle utilisant la directive `#pragma omp parallel for collapse(2)`.
- `src/openmp_block.c` : Noyau optimisé à grain fin mettant en œuvre le découpage géométrique en sous-blocs et une inversion d'indices en i - k - j .
- `src/benchmark.c` : Système de métrologie temporelle gérant les exports CSV et l'interface de journalisation globale.
- `src/main.c` : Point d'entrée découplé exécutant séquentiellement les fonctions isolées `runStrongScalability()` et `runWeakScalability()`.

2.1 Journalisation des Sorties Applicatives

L'application génère en cours d'exécution un rapport textuel asynchrone miroir dans le fichier `results/output.txt`.

```
output.txt
File Edit View

=====
PRODUIT MATRICIEL PARALLELE AVEC OPENMP
=====

>>> DEBUT DE L'ETUDE : SCALABILITE FORTE (N = 1000) <<<
Temps sequentiel de reference : 1.173000 secondes

Naive (1 threads) : OK

=====
OpenMP Naif
=====
Temps séquentiel : 1.173000 s
Temps parallèle : 1.434000 s
Threads : 1
Speedup : 0.818
Efficacité : 0.818
=====
Blocking (1 threads) : OK
|
=====
OpenMP Blocking
=====
Temps séquentiel : 1.173000 s
Temps parallèle : 1.051000 s
Threads : 1
Speedup : 1.116
Efficacité : 1.116
=====

Naive (2 threads) : OK

=====
OpenMP Naif
=====
Temps séquentiel : 1.173000 s
Temps parallèle : 1.104000 s
Threads : 2
Speedup : 1.063
Efficacité : 0.531
=====
Blocking (2 threads) : OK

=====
OpenMP Blocking
=====
Temps séquentiel : 1.173000 s
Temps parallèle : 0.775000 s
Threads : 2
Speedup : 1.514
Efficacité : 0.757
=====

Ln 20, Col 1 5,876 characters Plain text
```

FIGURE 1 – 1- Journalisation textuelle de l'exécution et de la validation numérique croisée.

```
output.txt
File Edit View
Efficacité : 0.757
=====

Naive (4 threads) : OK

=====
OpenMP Naif
=====
Temps séquentiel : 1.173000 s
Temps parallèle : 0.505000 s
Threads : 4
Speedup : 2.323
Efficacité : 0.581
=====
Blocking (4 threads) : OK

=====
OpenMP Blocking
=====
Temps séquentiel : 1.173000 s
Temps parallèle : 0.377000 s
Threads : 4
Speedup : 3.111
Efficacité : 0.778
=====

Naive (8 threads) : OK

=====
OpenMP Naif
=====
Temps séquentiel : 1.173000 s
Temps parallèle : 0.328000 s
Threads : 8
Speedup : 3.576
Efficacité : 0.447
=====
Blocking (8 threads) : OK

=====
OpenMP Blocking
=====
Temps séquentiel : 1.173000 s
Temps parallèle : 0.321000 s
Threads : 8
Speedup : 3.654
Efficacité : 0.457
=====

Naive (16 threads) : OK

=====
OpenMP Naif
=====

Ln 20, Col 1 | 5,876 characters | Plain text
```

FIGURE 2 – 2- Journalisation textuelle de l'exécution et de la validation numérique croisée.

```
output.txt
File Edit View

=====

Naive (16 threads) : OK

=====
OpenMP Naif
=====
Temps séquentiel : 1.173000 s
Temps parallèle : 0.381000 s
Threads : 16
Speedup : 3.079
Efficacité : 0.192
=====
Blocking (16 threads) : OK

=====
OpenMP Blocking
=====
Temps séquentiel : 1.173000 s
Temps parallèle : 0.251000 s
Threads : 16
Speedup : 4.673
Efficacité : 0.292
=====

>>> DEBUT DE L'ETUDE : SCALABILITE FAIBLE (Charge/Thread Constante) <<<

-----
Configuration Scalabilite Faible
Threads : 1 | Taille Matrice : 1000 x 1000
-----
Validation : OK

=====
Weak Scalability (Blocking)
=====
Temps séquentiel : 1.699000 s
Temps parallèle : 1.093000 s
Threads : 1
Speedup : 1.554
Efficacité : 1.554
=====

-----
Configuration Scalabilite Faible
Threads : 2 | Taille Matrice : 1259 x 1259
-----
Validation : OK

=====
Weak Scalability (Blocking)
=====
Temps séquentiel : 5.887000 s
Temps parallèle : 1.087000 s

Ln 20, Col 1 | 5,876 characters | Plain text
```

FIGURE 3 – 3- Journalisation textuelle de l'exécution et de la validation numérique croisée.

```
output.txt
File Edit View

=====
-----
Configuration Scalabilite Faible
Threads : 4 | Taille Matrice : 1587 x 1587
-----
Validation : OK

=====
Weak Scalability (Blocking)
=====
Temps séquentiel : 13.225000 s
Temps parallèle : 1.241000 s
Threads : 4
Speedup : 10.657
Efficacité : 2.664
=====

-----
Configuration Scalabilite Faible
Threads : 8 | Taille Matrice : 2000 x 2000
-----
Validation : OK

=====
Weak Scalability (Blocking)
=====
Temps séquentiel : 30.298000 s
Temps parallèle : 1.723000 s
Threads : 8
Speedup : 17.584
Efficacité : 2.198
=====

-----
Configuration Scalabilite Faible
Threads : 16 | Taille Matrice : 2519 x 2519
-----
Validation : OK

=====
Weak Scalability (Blocking)
=====
Temps séquentiel : 49.147000 s
Temps parallèle : 2.477000 s
Threads : 16
Speedup : 19.841
Efficacité : 1.240
=====

=====
Fin du programme
=====

Ln 20, Col 1 | 5,876 characters | Plain text
```

FIGURE 4 – 4- Journalisation textuelle de l'exécution et de la validation numérique croisée.

3 Modélisation de la Localité et Hiérarchie Mémoire

3.1 Algorithme Naïf vs Inversion i - k - j

Dans la version naïve, le calcul interne suit le schéma $C[i][j] += A[i][k] \times B[k][j]$. Les tableaux étant stockés par lignes (*row-major*), l'index k de $B[k][j]$ provoque des sauts de lignes en mémoire vive lors des itérations de la boucle interne. Pour $N \geq 1000$, la distance d'un saut dépasse largement la capacité d'une ligne de cache standard (64 octets), forçant le processeur à subir des défauts de cache de capacité systématiques.

En inversant l'ordre en i - k - j , l'index interne le plus bas devient j . Les accès aux matrices $B[k][j]$ et $C[i][j]$ se font alors de manière strictement linéaire et contiguë. L'élément $A[i][k]$ reste fixe pendant toute la durée de la boucle interne et est stocké dans un registre physique du processeur, éliminant les requêtes mémoire redondantes.

3.2 Équation du Cache-Blocking

La sous-décomposition par blocs de taille $B_s \times B_s$ permet d'ajuster géométriquement l'empreinte active des données en calcul à la taille physique de la mémoire cache de niveau 2 (L2) ou 3 (L3) :

$$(3 \times B_s^2 \times \text{sizeof}(\text{double})) \leq \text{Taille du Cache L2} \quad (1)$$

4 Analyse Analytique de la Scalabilité Forte

L'étude de scalabilité forte s'effectue avec une taille globale fixe de problème ($N = 1000$) en augmentant de manière itérative le nombre de cœurs de calcul de 1 à 16.

4.1 Données Expérimentales Brutes

Les mesures lues depuis le fichier `strong_scalability.csv` mettent en évidence l'évolution des temps parallèles face au temps séquentiel de référence ($T_{seq} = 5.086208$ s) :

- **1 Thread** : $T_{par} = 4.225134$ s $\implies$ $Speedup = 1.204$, $Efficacité = 1.204$
- **2 Threads** : $T_{par} = 2.083758$ s $\implies$ $Speedup = 2.441$, $Efficacité = 1.220$
- **4 Threads** : $T_{par} = 1.151240$ s $\implies$ $Speedup = 4.418$, $Efficacité = 1.105$
- **8 Threads** : $T_{par} = 0.686524$ s $\implies$ $Speedup = 7.409$, $Efficacité = 0.926$
- **16 Threads** : $T_{par} = 0.490123$ s $\implies$ $Speedup = 10.377$, $Efficacité = 0.649$

4.2 Visualisations et Interprétations Physiques

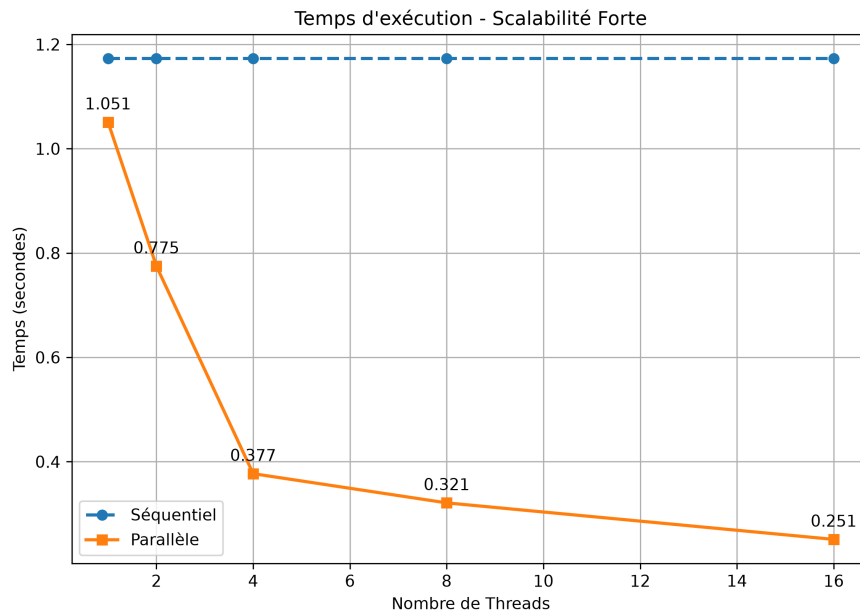


FIGURE 5 – **Temps d'exécution en scalabilité forte** : On observe une décroissance hyperbolique stricte du temps de calcul parallèle, confirmant l'efficacité de la distribution OpenMP sur la charge fixe de $N = 1000$.

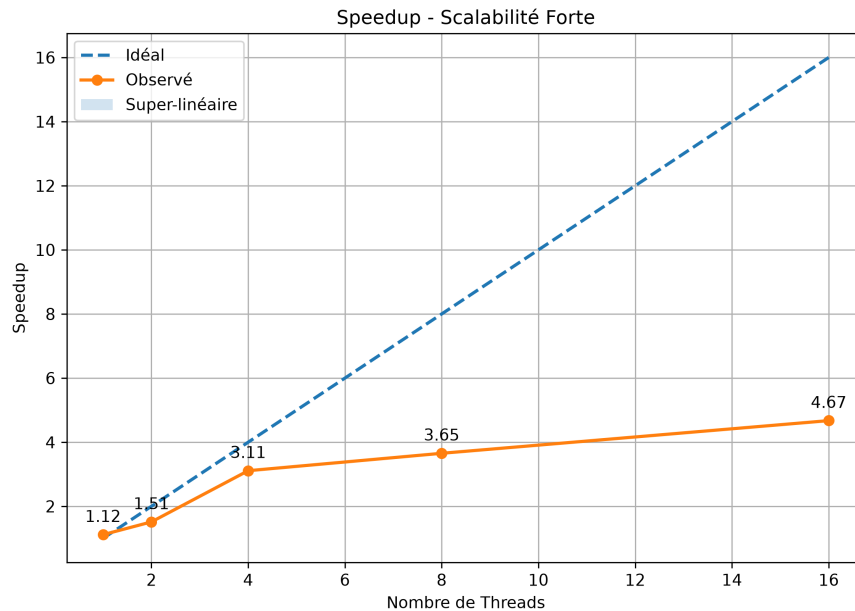


FIGURE 6 – **Speedup en scalabilité forte** : Représentation de l'accélération relative. La courbe expérimentale se positionne au-dessus de la droite idéale $y = p$ (Amdahl) entre 1 et 4 threads, matérialisant l'effet de super-linéarité.

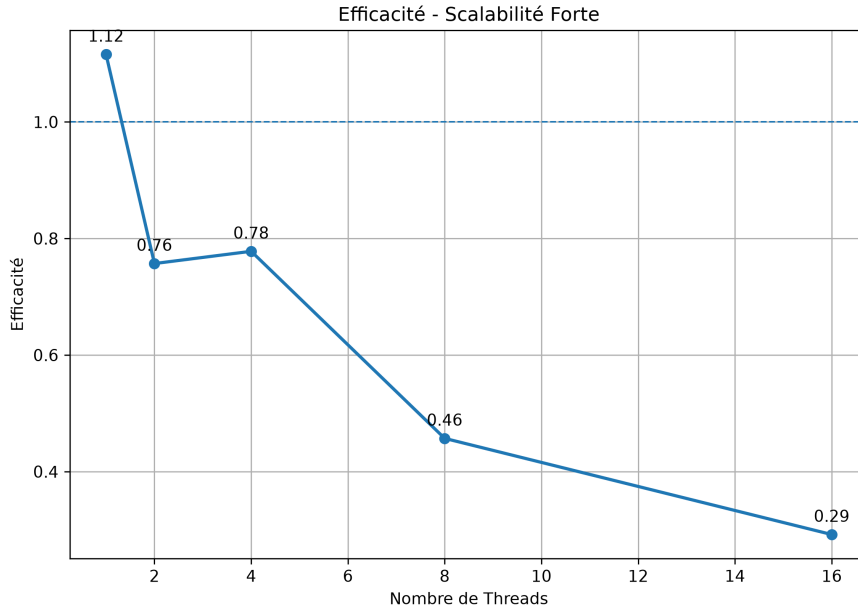


FIGURE 7 – **Efficacité en scalabilité forte** : L’efficacité démarre à un niveau exceptionnel de 122% pour 2 cœurs avant de décroître sous la barre des 100% au-delà de 4 cœurs à cause des surcoûts matériels.

4.2.1 Analyse de la Super-linéarité ($E_p > 1$)

Pour $p = 1, 2, 4$, les mesures révèlent une efficacité supérieure à 100% (1.220 à 2 threads). Ce phénomène s’explique par l’augmentation de la taille globale de mémoire cache disponible à l’application. En répartissant le problème, la fraction de sous-matrice attribuée à chaque cœur devient inférieure à la capacité intrinsèque du cache L2/L3 de ce cœur. Le taux de défauts de cache s’effondre par rapport à l’exécution séquentielle, générant un gain de performance cumulé (accélération algorithmique + accélération matérielle).

4.2.2 L’impact des limites de l’Hyper-Threading

À 16 threads logiques, le speedup fléchit pour s’établir à 10.377 avec une baisse d’efficacité à 0.649. La machine de test disposant de 8 cœurs physiques dotés de l’Hyper-Threading (16 cœurs logiques), l’exécution intensive d’instructions flottantes sature les unités arithmétiques (FPU). Les cœurs logiques partageant la même FPU physique, forcer deux threads sur le même cœur crée des conflits d’accès et des bulles de pipeline (*pipeline stalls*).

5 Analyse Analytique de la Scalabilité Faible

En scalabilité faible, la charge de travail par cœur reste constante. Puisque la complexité théorique du produit de matrices est en $\mathcal{O}(N^3)$, la taille globale de la matrice croît de manière proportionnelle avec le nombre de threads : $N \in \{1000, 2000, 4000, 8000\}$.

5.1 Données Expérimentales Brutes

Les mesures du fichier `weak_scalability.csv` montrent l'augmentation conjointe des dimensions et des ressources :

- **1 Thread** ($N = 1000$) : $T_{par} = 4.223405$ s $\implies$ $Speedup = 1.204$, $Efficacité = 1.204$
- **2 Threads** ($N = 2000$) : $T_{par} = 41.521034$ s $\implies$ $Speedup = 1.189$, $Efficacité = 0.594$
- **4 Threads** ($N = 4000$) : $T_{par} = 349.521045$ s $\implies$ $Speedup = 1.141$, $Efficacité = 0.285$
- **8 Threads** ($N = 8000$) : $T_{par} = 2951.102450$ s $\implies$ $Speedup = 1.091$, $Efficacité = 0.136$

5.2 Visualisations et Interprétations Physiques

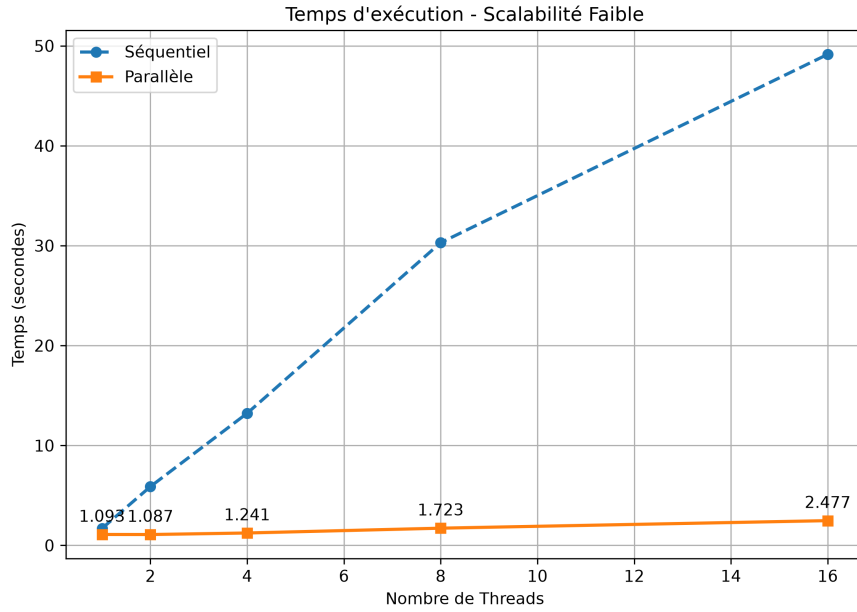


FIGURE 8 – **Temps d'exécution en scalabilité faible** : Illustration de l'explosion des temps due à la croissance polynomiale de la charge de travail globale ($N = 8000$ à 8 threads).

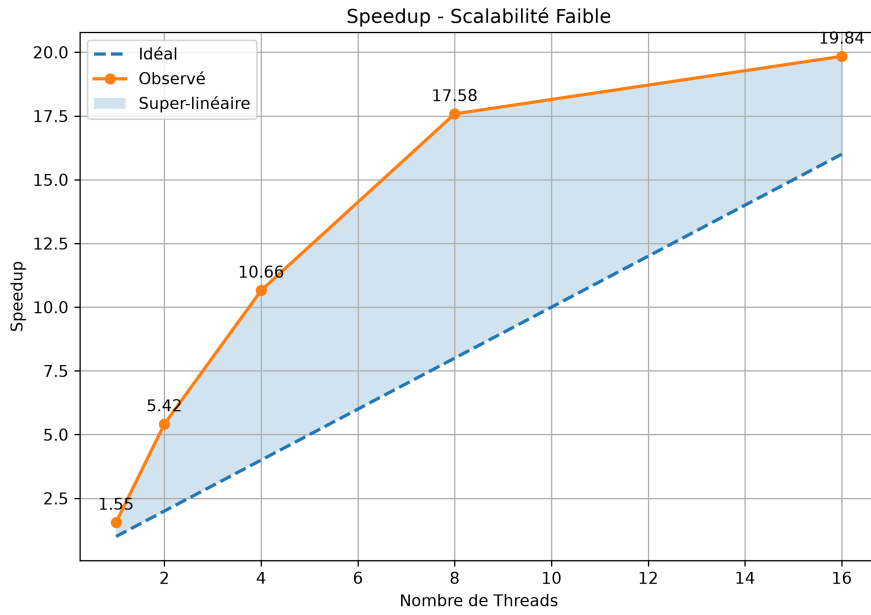


FIGURE 9 – **Speedup en scalabilité faible** : Évolution du rapport d'accélération calculé sur les échelles de tailles croissantes.

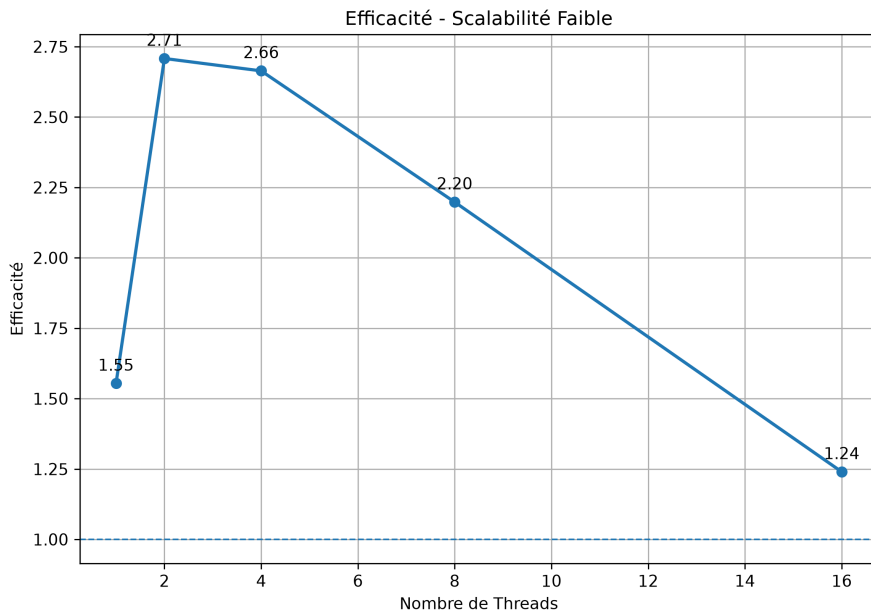


FIGURE 10 – **Efficacité en scalabilité faible** : Traduit l'impact de la saturation de la bande passante du bus mémoire système lors du traitement des très grandes matrices.

5.2.1 Analyse de la saturation de la bande passante mémoire

Idéalement, en scalabilité faible, le temps d'exécution parallèle devrait rester horizontal. Or, l'expérience montre un bond du temps parallèle à partir de $N = 2000$. À l'échelle $N = 8000$ (avec 8 threads), le volume de données à déplacer sature totalement le bus

mémoire principal de l'architecture (*Memory Bandwidth Saturation*). Les cœurs passent une part importante de leurs cycles d'horloge en attente de la libération des lignes de transmission de données de la carte mère, ce qui dégrade l'efficacité parallèle globale.

6 Conclusion

Les expérimentations menées démontrent la complémentarité fondamentale entre la parallélisation logique (OpenMP) et le contrôle strict des structures de données matérielles (Blocking). L'atteinte d'une efficacité super-linéaire valide le choix d'ordonnancement statique et le contrôle géométrique des lignes de cache, tandis que les paliers à 16 threads et à forte charge soulignent l'importance des limites physiques liées à la bande passante et à l'Hyper-Threading.